

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

LUIZ EDUARDO FERREIRA ANSILIERO

TRABALHO FINAL

**GUARAPUAVA-PR
2026**

Trabalho Final

1. print da execução dos exercícios;
2. resposta a(s) pergunta(s), se tiver;
3. o que você entendeu executando o comando.

Resumo da parte teórica:

O processo no Linux é uma abstração que representa uma instância de um programa em execução, podendo ser uma tarefa do sistema ou do usuário.

Durante seu ciclo de vida, ele consome recursos essenciais como o processador, espaço em memória RAM e dispositivos de entrada e saída. Esses processos são classificados como interativos, quando exigem intervenção constante do usuário, batch, para execuções em lote via scripts, ou de tempo real, que possuem prioridade máxima sobre os demais em aplicações críticas.

Para gerenciar os recursos de forma justa, o sistema monitora se o processo está nos estados de espera (bloqueado), pronto, executando ou suspenso. No terminal, o comando **top** exibe essas informações de forma dinâmica e em tempo real, enquanto o comando **ps** fornece uma visão estática dos processos ativos.

Através do **pidof**, o usuário localiza o identificador (PID) de um processo para então controlá-lo com o comando kill, que permite enviar sinais como o **-STOP** para paralisar a execução, o **-CONT** para retomá-la ou o **-9** para encerrar o processo obrigatoriamente.

Além da execução em primeiro plano, é possível rodar programas em segundo plano (background) utilizando o caractere **&**, gerenciando-os posteriormente com os comandos **jobs**, **bg** (para enviar ao fundo) e **fg** (para trazer à frente).

Os comandos nice e renice permitem ajustar a prioridade de um processo na criação ou durante sua execução, respectivamente, sendo que apenas o administrador pode atribuir as prioridades mais altas (valores negativos). A prioridade no Linux é organizada em uma faixa de 40 valores (de -20 a 19), onde um valor numérico mais baixo indica uma maior prioridade de execução.

Exercício A:

1:

```
luiz@luiz-Vivobook-Go-E1504FA-E1504FA: ~
top - 19:29:21 up 9:30, 1 user, load average: 1,27, 1,43, 1,36
Tarefas: 390 total, 2 em exec., 387 dormindo, 0 parado, 1 zumbi
%CPU(s): 3,3 us, 1,5 sy, 0,0 ni, 95,0 id, 0,0 wa, 0,0 hi, 0,2 si, 0,0 st
MB mem : 15262,6 total, 3048,0 free, 6335,9 used, 6393,5 buff/cache
MB swap: 2048,0 total, 2048,0 free, 0,0 used. 8926,7 avail mem
```

PID	USUARIO	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TEMPO+	COMANDO
1911	luiz	20	0	5399200	441848	142192	S	14,3	2,8	34:33.27	cinnamon
2182	luiz	20	0	2772244	226336	108616	S	8,6	1,4	24:01.52	hidamar+
1358	root	20	0	1675548	199280	106916	R	3,3	1,3	18:05.50	Xorg
306043	luiz	20	0	377544	26592	20484	S	2,7	0,2	0:00.08	gnome-s+
253955	luiz	20	0	1419,9g	523252	178452	S	1,7	3,3	3:52.12	Discord
3452	luiz	20	0	631456	126020	47936	S	1,3	0,8	1:09.34	mintrep+
281295	root	20	0	0	0	0	I	1,3	0,0	0:05.09	kworker+
1542	luiz	9	-11	133608	30532	10480	S	1,0	0,2	2:13.85	pipewir+
525	root	-51	0	0	0	0	S	0,7	0,0	2:11.75	irq/79-+
1538	luiz	9	-11	118800	16820	9428	S	0,7	0,1	1:19.42	pipewire
216071	luiz	20	0	52,4g	145312	112324	S	0,7	0,9	0:26.10	chrome
293611	root	20	0	0	0	0	I	0,7	0,0	0:02.69	kworker+
15	root	20	0	0	0	0	I	0,3	0,0	0:42.13	rcu_pre+
998	root	20	0	1355716	44164	25540	S	0,3	0,3	0:07.10	tailsca+
1739	luiz	20	0	11672	7244	4596	S	0,3	0,0	0:21.94	dbus-da+
193952	luiz	20	0	755696	110764	38384	S	0,3	0,7	0:08.65	gnome-t+
216021	luiz	20	0	53,0g	464900	277436	S	0,3	3,0	1:22.30	chrome

2:

Linha 1 (Tempo): Horário atual, tempo ligado (uptime), usuários logados e média de carga (load average) em 1, 5 e 15 minutos;

Linha 2 (Tarefas): Total de tarefas em execução, dormindo, paradas e zumbis;

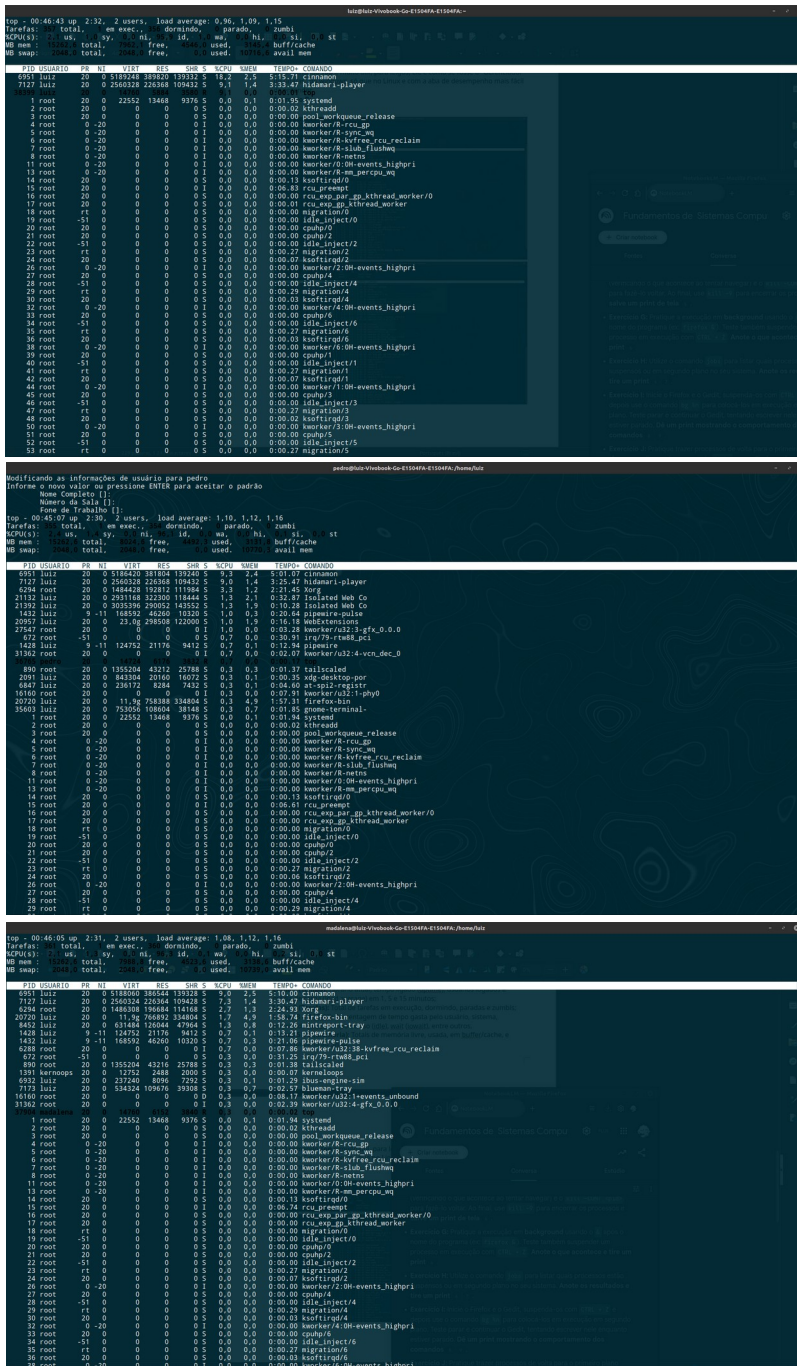
Linhas 3 e 4 (CPU): Porcentagem de tempo gasta pelo usuário, sistema, prioridade (nice), tempo ocioso (idle), wait (iowait), entre outros.

Linhas 5 e 6 (Memória): Totais de memória livre, usada, em buffer/cache, e status da partição Swap.

3: O comando **Top** é muito útil, para leigos, ele é como se fosse o “Gerenciador de tarefas” do windows, só que no Linux e com a aba de desempenho mais fácil de entender

Exercício B:

1:



- 2:
- 3: Eu criei os usuários utilizando “adduser”, depois de entrar no terminal root usando “sudo su”, e para mudar entre os usuários usei “su <usuario>”

Exercício C:

1:

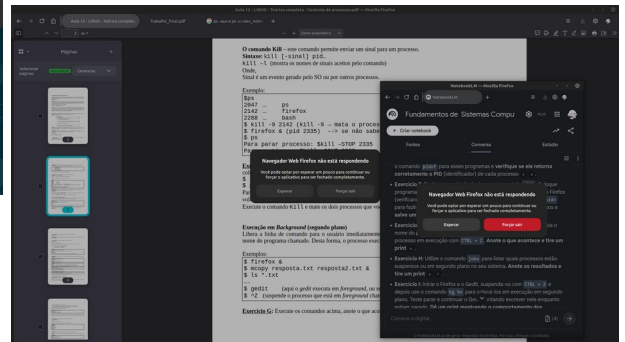

```
luiz@luiz-Vivobook-Go-E1504FA-E1504FA:~$ pidof firefox
20720
luiz@luiz-Vivobook-Go-E1504FA-E1504FA:~$ pidof gedit
63347
luiz@luiz-Vivobook-Go-E1504FA-E1504FA:~$ pidof pico
63231
```

- 2: O retorno é o PID de cada processo, Ex: firefox: 20720.
- 3: O comando pidof serve para pegar o PID de um processo específico, facilitando ações direcionadas ao tal processo sem listar demais processos.

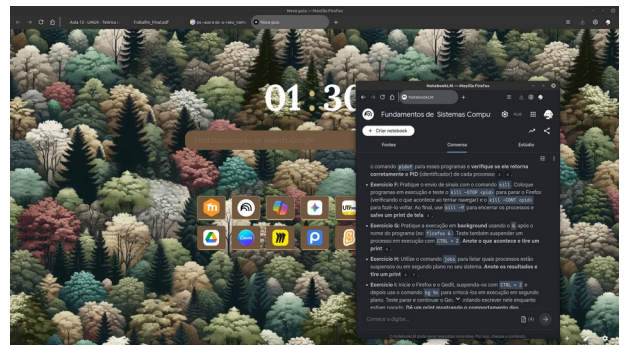
Exercício F:

1:

```
luiz@luiz-Vivobook-Go-E1504FA-E1504FA:~$ kill -STOP 20720
luiz@luiz-Vivobook-Go-E1504FA-E1504FA:~$ |
```



```
luiz@luiz-Vivobook-Go-E1504FA-E1504FA:~$ kill -CONT 20720
luiz@luiz-Vivobook-Go-E1504FA-E1504FA:~$ |
```

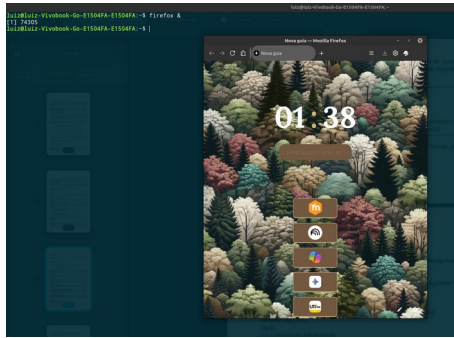


```
luiz@luiz-Vivobook-Go-E1504FA-E1504FA:~$ kill -9 70250
luiz@luiz-Vivobook-Go-E1504FA-E1504FA:~$ |
luiz@luiz-Vivobook-Go-E1504FA-E1504FA:~$ kill -9 70250 70249 70164 70163 70144 70132 70076 69994 69992 69957 69955 69954 69948
bash: kill: (70250) - Processo inexistente
luiz@luiz-Vivobook-Go-E1504FA-E1504FA:~$ |
```

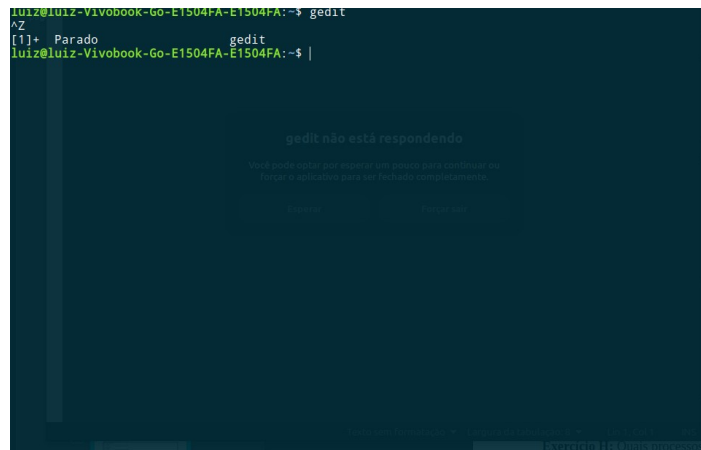
- 2:
- 3: O comando Kill pode pausar, eliminar e retornar processos ao funcionamento. Pode corrigir erros e até ser usado para parar bugs causados por apps problemáticos.

Exercício G:

1:



```
luiz@luiz-Vivobook-Go-E1504FA-E1504FA:~$ pico resposta.txt
[1]+  Concluído                  firefox
luiz@luiz-Vivobook-Go-E1504FA-E1504FA:~$ mcopy resposta.txt resposta2.txt &
[1] 75288
luiz@luiz-Vivobook-Go-E1504FA-E1504FA:~$ ls *.txt
resposta2.txt  resposta.txt  telefones.txt
[1]+  Concluído                  mcopy resposta.txt resposta2.txt
luiz@luiz-Vivobook-Go-E1504FA-E1504FA:~$ |
```



2: O **&** permite fazer mais de um processo ao mesmo tempo no terminal, deve ser útil especialmente para comandos como o **find** ou o **search** for que demoram para terminar.

O **CTRL + Z (^Z)** permite suspender processos, de forma semelhante ao **kill - STOP**.

3: Ambos são muito úteis, em especial a função de segundo plano do **&** é excelente pois permite usar o mesmo terminal para fazer ações mais lentas sem deixar de lado demais ações.

Exercício H:

1:

```
luiz@luiz-Vivobook-Go-E1504FA-E1504FA:~$ jobs
[1]+  Parado                  gedit
luiz@luiz-Vivobook-Go-E1504FA-E1504FA:~$ |
```

2: O **jobs** mostra processos suspensos.

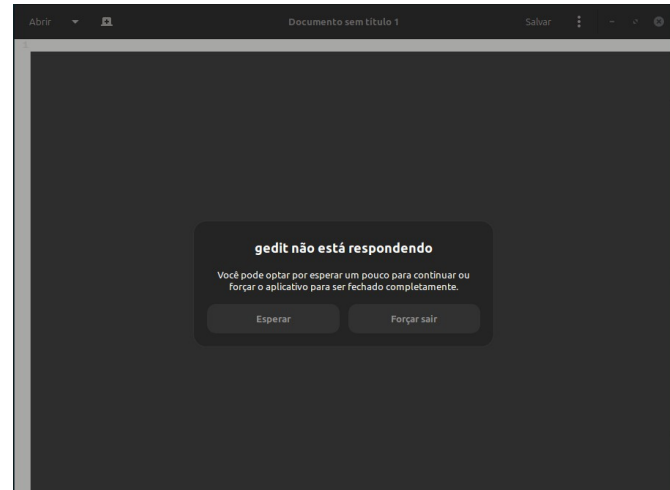
3: Extremamente necessário caso se use de suspender vários processos.

Exercício I:

1:



```
luiz@luiz-Vivobook-Go-E1504FA-E1504FA:~$ jobs
[2]-  Parado                  gedit
[3]+  Parado                  gedit
luiz@luiz-Vivobook-Go-E1504FA-E1504FA:~$ bg %2
[2]-  gedit &
luiz@luiz-Vivobook-Go-E1504FA-E1504FA:~$ bg %3
[3]+  gedit &
[2]   Concluido              gedit
Failed to register: O tempo limite foi alcançado
luiz@luiz-Vivobook-Go-E1504FA-E1504FA:~$ |
```



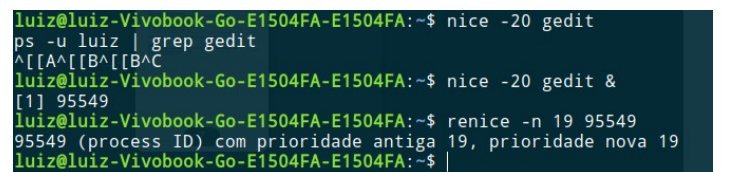
2: O comando `bg %[n]` consegue transferir processos suspensos diretamente para o segundo plano.

3: Evita tranquilamente o trabalho de trazer algo que está em suspensão para ativo para só então colocar em segundo plano.

Exercício J:

1:

```
luiz@luiz-Vivobook-Go-E1504FA-E1504FA:~$ gedit
^Z
[1]+  Parado                  gedit
luiz@luiz-Vivobook-Go-E1504FA-E1504FA:~$ bg %1
[1]+  gedit &
luiz@luiz-Vivobook-Go-E1504FA-E1504FA:~$ fg %1
gedit
```



2:
3: